

What is Inheritance?

Inheritance is a fundamental ObjectOrientedProgramming concept where a new class is created from an existing class. The subclass inherits properties and behaviors from its superclass while being able to add new features or modify inherited ones.

Key characteristics:

- **"Is-a" relationship** (e.g., a Dog is an Animal)
- Promotes **code reuse**
- Enables **polymorphism**

Modeling real-world hierarchies:

```
class Vehicle {}  
class Car extends Vehicle {}  
class ElectricCar extends Car {}
```

Importance of Inheritance

Without inheritance we would:

- **Duplicate code** for similar classes
- **Lose polymorphism** (ability to treat specialized objects as general ones)
- **Make maintenance harder** (changes would need to be made in multiple places)
- **Have less expressive code** (couldn't model natural hierarchies)

UML Diagram

Methods: + public, - private, # protected

Arrow: Solid line with hollow triangle (Child → Parent)

Overriding vs. Overloading

Overriding (runtime polymorphism): Same signatures

Overloading (compile-time polymorphism): Same method different parameters

1. **Overriding** requires:
 - Same method name
 - Same parameter types **in the same order**
 - Appears in a subclass
2. **Overloading** requires:
 - Same method name
 - Different parameter types **or order**

Give an example Make sure **EMPHASIZE SIGNATURE**

The Square Class

```
public class Square extends Rectangle  
{
```

Object Class

The Object class contains a few useful methods, which are inherited by all classes

I.e., the toString() method is defined in the Object class

Every time we define the toString method, we are actually **overriding** an inherited definition

The toString method in the Object class is defined to return a string that contains the name of the objects class along with some other information

Super: When a child class overrides a method or hides a field, super allows access to the parent's version.

- super() must be the **first statement** in a child constructor (if used).

```
Public void makeSound() {  
    super.makeSound(); // Calls Animal's makeSound()  
    System.out.println(super.sound); // Accesses Animal's 'sound'  
    System.out.println(sound); // Accesses Dog's 'sound' ("Bark")  
}
```

What is Polymorphism?

Polymorphism (Greek for "many forms") is an OOP concept where a single interface can represent different underlying forms. In Java, it allows objects of different classes to be treated as objects of a common superclass.

Example:

```
class Animal {  
    void makeSound() {  
        System.out.println("Animal sound");  
    }  
}  
  
class Dog extends Animal {  
    @Override  
    void makeSound() {  
        System.out.println("Bark");  
    }  
}  
  
class Cat extends Animal {  
    @Override  
    void makeSound() {  
        System.out.println("Meow");  
    }  
}
```

```

    }
}

public class Main {
    public static void main(String[] args) {
        Animal myAnimal = new Animal(); // Animal object
        Animal myDog = new Dog();       // Dog object
        Animal myCat = new Cat();        // Cat object

        myAnimal.makeSound(); // Output: Animal sound
        myDog.makeSound();    // Output: Bark
        myCat.makeSound();    // Output: Meow
    }
}

```

generic code that works with multiple types

Abstract Classes:

- Classes that cannot be instantiated
- May contain abstract methods (without implementation) and concrete methods
- Represent an "is-a" relationship

Interfaces:

- Completely abstract types
- Define contracts that implementing classes must follow
- Represent "can-do" relationships
- Comparable Interface!

Similarities

- Both cannot be instantiated directly
- Both are used to achieve abstraction
- Both can be used to achieve polymorphism
- Both can contain abstract methods

Differences

Feature	Abstract Class	Interface
Instantiation	Cannot be instantiated	Cannot be instantiated
Methods	Can have abstract and concrete	All methods were abstract (pre-Java 8)
Variables	Can have any variables	Only public static final
Constructors	Can have constructors	No constructors
Multiple Inheritance	Single inheritance	Multiple implementations

Use Abstract Classes When:

- You want to share code among closely related classes
- You need to define a template with some implemented methods
- You need a constructor

Use Interfaces When:

- You want to define a contract for unrelated classes
- You need multiple inheritance of type
- You want to specify what a class can do (behavior)

Sorting Algorithms:

Sorting is the process of arranging a group of n elements into a defined order based on particular criteria

Selection Sort:

SelectionSort orders values by repeatedly putting a particular value into its final sorted position

The algorithm:

1. find the smallest value in the dataset
2. switch it with the value in the first position
3. find the next smallest value in the dataset
4. switch it with the value in the second position
5. repeat until all values are in their sorted positions

InsertionSort orders a list of values by repeatedly inserting a particular value into a sorted subset of the list

The algorithm:

1. consider the first item to be a sorted sublist of length 1
2. insert the second item in the sorted sublist,
3. shifting the first item if needed
4. insert the third item into the sorted sublist,
5. shifting the other items as needed
6. repeat until all values inserted into their proper positions

Merge sort orders a list of values by recursively dividing the list in half until each sublist has one element, then recombining (merging) the sublists

The algorithm:

Split: Divide the list into two equal parts—left and right. If the list has odd length, one half will have an extra element.

Sort: Recursively use merge-sort to sort the two lists. Note that by definition, a list of one element is sorted (base case).

Merge: Merge the two sorted parts into one sorted list

Algorithm	Best	Average	Worst	Space in addition to original array
Insertion	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Merge	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$

Java Generics

What are Java Generics?

Generics enable types (classes and interfaces) to be parameters when defining classes, interfaces, and methods. They allow for type safety and eliminate the need for explicit type casting.

In Java, T is often used as a generic type parameter, most commonly seen in generic classes, methods, or interfaces. It stands for "Type", and it's a placeholder for any object type.

Importance of Generics

Without generics we would:

- Need to use Object types and explicit casting everywhere
- Lose compile-time type safety (runtime errors instead of compile-time errors)
- Have less readable code (full of type casts)
- Be unable to create type-safe collections

LinkedList vs Arrays

Benefits of LinkedLists - take advantage when working with one side of the list

- **Dynamic size:** Grows/shrinks as needed (no fixed capacity)
- **Efficient insertions/deletions:** $O(1)$ at head, $O(n)$ elsewhere (no shifting needed)

Benefits of Arrays

- **Random access:** $O(1)$ access to any element
- **Less overhead:** No need for node objects/pointers

Time Complexity Comparison

Operation	Array	Linked List
Access	$O(1)$	$O(n)$ or $O(1)$ for first node
Insert at head	$O(n)$	$O(1)$
Insert at tail	$O(1)^*$	$O(n)$
Delete at head	$O(n)$	$O(1)$
Search	$O(n)$	$O(n)$

*Amortized $O(1)$ for dynamic arrays

Linked Lists work great for things that operate on the front of a list.